Cap Token Review

A smart contract assessment of Cap token contract was performed by Octane Security, with a focus on the security aspects of the application's implementation.

About Octane Security

Octane Security is an AI-enhanced blockchain security firm dedicated to safeguarding digital assets. We combine security research with cutting-edge machine learning models to maximize protocol security and minimize risk. Our team ranks in the top of audit contest leaderboards, captures bug bounties, and has served big-name clients such as Alchemy, Optimism, Notional Finance, Blast, and Graph Protocol in the past.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We also rely on information that is provided by the client, its affiliates and partners for vulnerability identification. We can not guarantee the absense of vulnerabilities, issues, flaws, or defects after the review. It is up to the client to decide whether to implement recommendations and we take no liability for invalid recommendations. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Severity Matrix

Impact \ Likelihood	**Low**	**Medium**	**High**
Low	I	L	M
Medium	L	M	H
High	M	H	C

Security Assessment Summary

The scope of this review was limited to files at commit: 0b6701365eb7d589cf3fc9083a2ab742c6cd500a

Scope

The following smart contracts were in scope of the audit:

- Token.sol

Findings

Finding Number	Finding Title
I-1	Ownership Renouncement Can Permanently Disable Upgrades

I-1 Ownership Renouncement Can Permanently Disable Upgrades

Description

The token includes an upgradeability mechanism that can only be triggered by the owner:

```
function _authorizeUpgrade(address) internal view override onlyOwner { }
```

However, the owner can renounce ownership by calling `renounceOwnership` in `OwnableUpgradeable.sol`, which transfers ownership to the zero address:

```
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

After ownership is renounced, there is no longer an owner, meaning the upgrading functionality becomes permanently unavailable.

Recommendation

To avoid this risk, consider one of the following approaches:

- Disable ownership renouncement by overriding `renounceOwnership` and reverting in its implementation.
- Clearly document that the owner must never renounce ownership.

Resolution